



CompSci 130

Introduction to Software Fundamentals

Classes and Objects



Agenda & Reading

► Topics:

- Defining your own Python classes
- Classes and instances (objects)
- Defining new methods for a class
- Overriding default behaviour (methods)

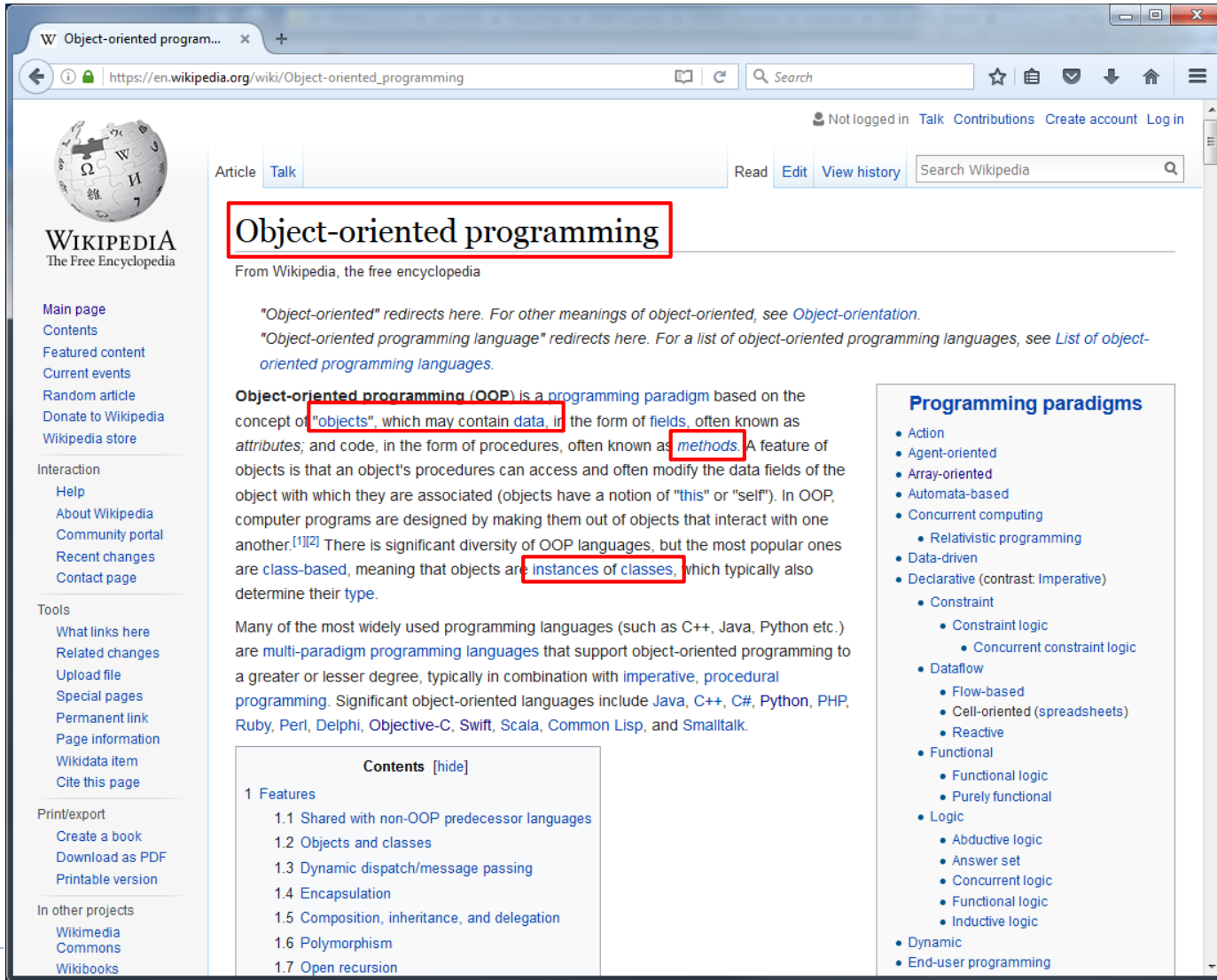
class Car:



► Further Study:

- Online Textbook: Chapter 5
 - <https://onlinetextbook.auckland.ac.nz/5-classes-and-objects/>

Why classes?



The screenshot shows the Wikipedia page for "Object-oriented programming". The title "Object-oriented programming" is highlighted with a red box. The page content includes a definition of OOP, a list of programming paradigms, and a table of contents. Several terms are highlighted with red boxes: "objects", "data", "fields", "methods", "class-based", and "instances of classes".

W Object-oriented program... x +

https://en.wikipedia.org/wiki/Object-oriented_programming

Not logged in Talk Contributions Create account Log in

Article Talk

Read Edit View history Search Wikipedia

Object-oriented programming

From Wikipedia, the free encyclopedia

"Object-oriented" redirects here. For other meanings of object-oriented, see [Object-orientation](#).

"Object-oriented programming language" redirects here. For a list of object-oriented programming languages, see [List of object-oriented programming languages](#).

Object-oriented programming (OOP) is a programming paradigm based on the concept of **"objects"**, which may contain **data**, in the form of **fields**, often known as **attributes**; and code, in the form of procedures, often known as **methods**. A feature of objects is that an object's procedures can access and often modify the data fields of the object with which they are associated (objects have a notion of **"this"** or **"self"**). In OOP, computer programs are designed by making them out of objects that interact with one another.^{[1][2]} There is significant diversity of OOP languages, but the most popular ones are **class-based**, meaning that objects are **instances of classes**, which typically also determine their type.

Many of the most widely used programming languages (such as C++, Java, Python etc.) are multi-paradigm programming languages that support object-oriented programming to a greater or lesser degree, typically in combination with **imperative**, **procedural programming**. Significant object-oriented languages include **Java**, **C++**, **C#**, **Python**, **PHP**, **Ruby**, **Perl**, **Delphi**, **Objective-C**, **Swift**, **Scala**, **Common Lisp**, and **Smalltalk**.

Contents [hide]

- Features
 - Shared with non-OOP predecessor languages
 - Objects and classes
 - Dynamic dispatch/message passing
 - Encapsulation
 - Composition, inheritance, and delegation
 - Polymorphism
 - Open recursion

Programming paradigms

- Action
- Agent-oriented
- Array-oriented
- Automata-based
- Concurrent computing
 - Relativistic programming
- Data-driven
- Declarative (contrast: Imperative)
 - Constraint
 - Constraint logic
 - Concurrent constraint logic
- Dataflow
 - Flow-based
 - Cell-oriented (spreadsheets)
 - Reactive
- Functional
 - Functional logic
 - Purely functional
- Logic
 - Abductive logic
 - Answer set
 - Concurrent logic
 - Functional logic
 - Inductive logic
- Dynamic
- End-user programming



Why classes?

- ▶ Previously seen:
 - ▶ Data objects getting created in code. Functions are used to process this data. This has benefits of (1) reducing code duplication and (2) hiding away implementation details.
 - ▶ We call this hiding away of details **encapsulation**.
- ▶ Classes and objects follow this paradigm of encapsulation one step further:
 - ▶ Data (state) and functions (called methods) working with this data are encapsulated in **objects**.
 - ▶ Classes are the blueprints for such objects. Objects are instantiations of classes.
 - ▶ Remember: In Python everything is an object!
 - ▶ Python is an object-oriented programming (OOP) language.

Object-oriented programming

- ▶ Here are two objects (from the “real world”)



- ▶ They are both the same kind of thing (a car)  **class**
- ▶ So they both have the same kinds of attributes  **state**
- ▶ They can also “do” the same kinds of actions  **behavior**

Object-oriented programming

- ▶ Here are two objects of different kinds:



`blue_container.open_door()`



`red_car.brake()`

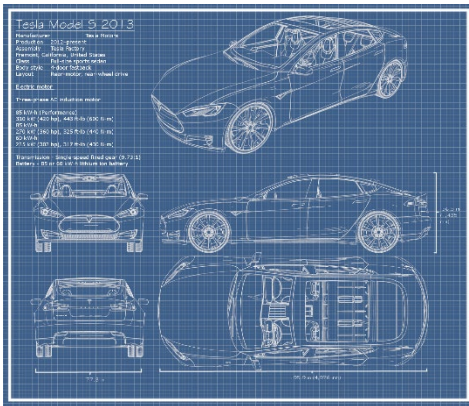
In object-oriented programming, we try to model the problem we are solving using **objects**. We can have many objects of different kinds (**classes**). At any point in time, every object in our program has some particular **states** (a set of variables storing data attributes for that object). We can perform actions on the objects in our program by calling **methods** (functions) on them. Objects bundle data & methods.

Classes vs. objects

- ▶ A class defines a category of objects with its state & behaviour

Blueprint

Building new cars



State



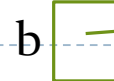
State

Shared behaviour
(e.g. accelerate, brake, refuel, ...)

```
class Car:  
    ...  
    ...
```

Car class

```
a = Car()  
b = Car()
```



Car objects

Instances of the Car class



Classes

▶ Python has a number of built-in classes

▶ list, dict, set, int, float, bool, str, ...

▶ We can define our own classes

▶ creates a new **type** of Python object!

▶ to check for types:

class *name_of_the_class*:

definition of the class goes here

initializer

methods

state variables

```
a = [10, 20, 30]
b = {'a': 5, 'b': 10}
c = 10
d = 10.55
e = True
f = 'Hello'
g = Car()
```



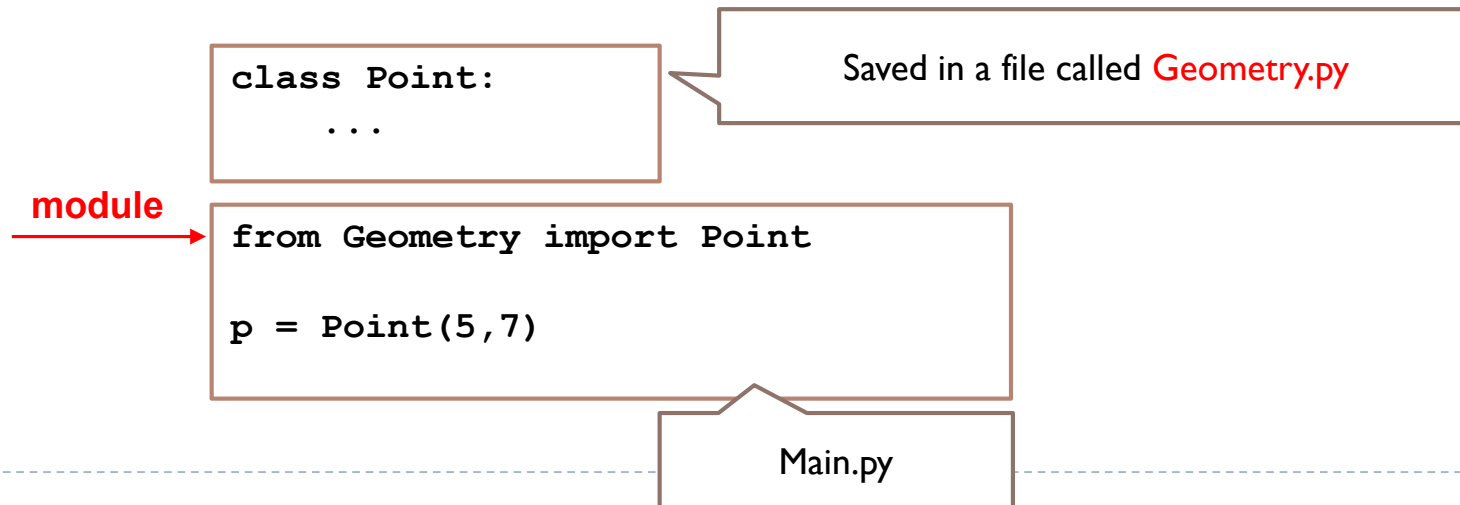
```
print(type(a), type(b), type(c))
print(type(d), type(e), type(f))
print(type(g))
```

```
<class 'list'>
<class 'dict'>
<class 'int'>
<class 'float'>
<class 'bool'>
<class 'str'>
<class 'Car'>
```




Saving the class

- ▶ Classes are designed to help building modular code
 - ▶ Classes can be defined within a **module** (i.e. a file) that also contains application code, or they can be defined in separate files (preferred).
 - ▶ Multiple classes can be defined in the same file, structuring them.
- ▶ In this course, we will store each class in its own module
 - ▶ To use the class in another module, you will need to **import** the module





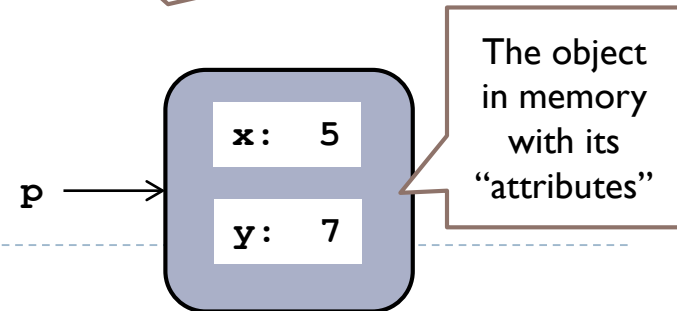
Initializers / Constructors

- ▶ Each class should contain a method to initialize objects
 - ▶ The name of the method is `__init__`
 - ▶ Other languages use constructors, technically `__init__` is an initializer
 - ▶ The method always has at least one parameter, called “self”
 - ▶ “self” is a reference to the object that we are creating
 - ▶ The initializer can (and frequently will) have other parameters

```
class Point:  
    def __init__(self, x, y):  
        self.x = x  
        self.y = y
```

```
from Geometry import Point  
  
p = Point(5, 7)
```

It creates an object in the memory for the class.





Accessing objects

- ▶ After an object is created, you can access its data fields and invoke its methods using the dot operator (.)
 - ▶ also known as the object member access operator
- ▶ For example, the following code accesses the x, y coordinates

```
from Geometry import Point
```

```
p = Point(5, 7)
```

```
print(p.x)
```

```
print(p.y)
```

5

7

```
p2 = Point(10, 20)
```

```
print(p2.x)
```

```
print(p2.y)
```

10

20



Adding functionality

► Defining more methods

- A method to shift a point geometrically by a given distance in horizontal (dx) and vertical (dy) directions.

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def translate(self, dx, dy):
        self.x += dx
        self.y += dy
```

main.py

```
from Geometry import Point

p = Point(5, 7)
print(p.x, p.y)

p.translate(1, 0)
print(p.x, p.y)
```

- Note: The method is named like a normal function, but always has the additional parameter (**self**) as **first** parameter.
- All methods that are called on an instance of an object need the “self” parameter!



Adding functionality

Geometry.py

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def translate(self, dx, dy):
        self.x += dx
        self.y += dy
```

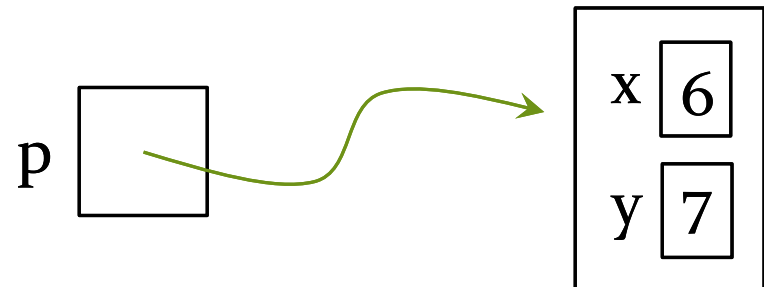
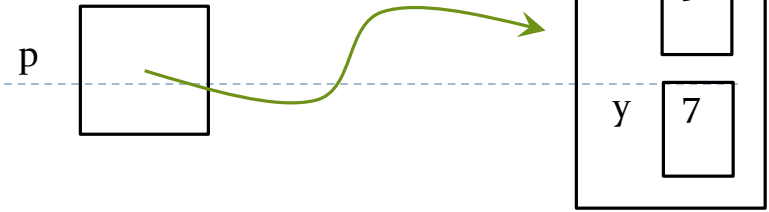
```
from Geometry import Point
```

```
p = Point(5, 7)
print(p.x, p.y)
```

```
p.translate(1, 0)
print(p.x, p.y)
```

main.py

```
p.translate(1, 0)
print(p.x, p.y)
```





How does this work?

- ▶ When you call a method like this:

`p.translate(1, 0)`

- ▶ The method call translates to:

`Point.translate(p, 1, 0)`

This is the special
“self” parameter

- ▶ Each object is an instance of a class (here Point)
- ▶ “self” is a parameter that *represents* the actual object instance
 - ▶ It is used to access specific object variables (attributes, data fields) inside the methods
 - ▶ Each object has its own instance variables. For example, the syntax **self.x** accesses the instance variable **x** for the specific object in a method.

Exercise 1

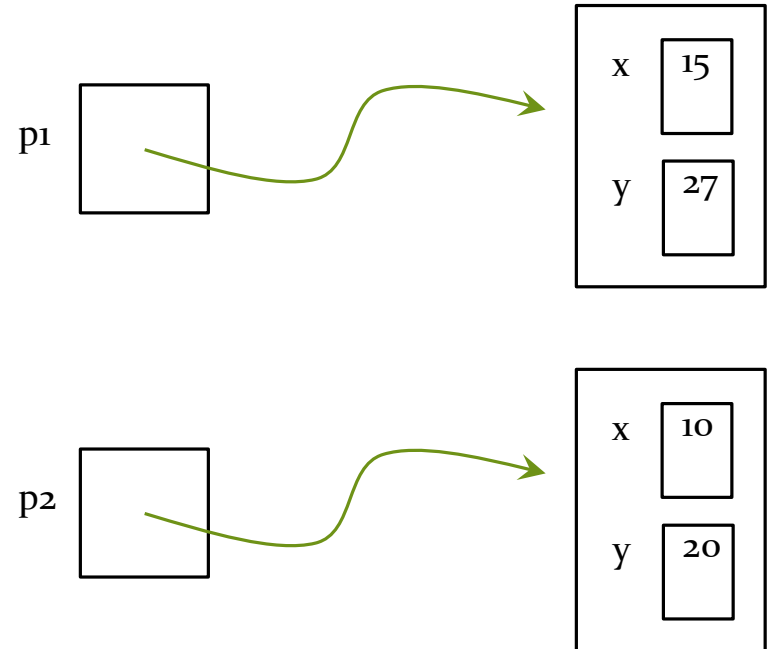
- What is the output of the following code?

```
from Geometry import Point

p1 = Point(15, 27)
p2 = Point(10, 20)
print(p1.x, end = ' ')
print(p2.y, end = ' ')

p1.translate(10, 20)
print(p1.x, p1.y, end = ' ')

p2.translate(20, 30)
print(p1.x, p1.y, end = ' ')
```



Exercise 2

- ▶ Define a class named **Square**. It contains
 - ▶ A private data field named *side_length* that defines the side length of a square.
 - ▶ A constructor/initializer that accepts a number representing the side length of a square
 - ▶ A method named *get_perimeter()* which returns the perimeter of a square.

```
from Geometry import Square
```

```
s = Square(10)
```

```
print(s.get_perimeter())
```

40



Example: A class for fractions

- ▶ Consider writing a class to represent a fraction in Python
- ▶ **Interface** to a client (user) of the class:
 - ▶ Create a fraction
 - ▶ Add, subtract, multiply, divide two fractions
 - ▶ Display a text representation of a fraction

Geometry.py

```
class Fraction:  
    ...
```

main.py

```
from Geometry import Fraction  
  
f = Fraction(1, 2)
```

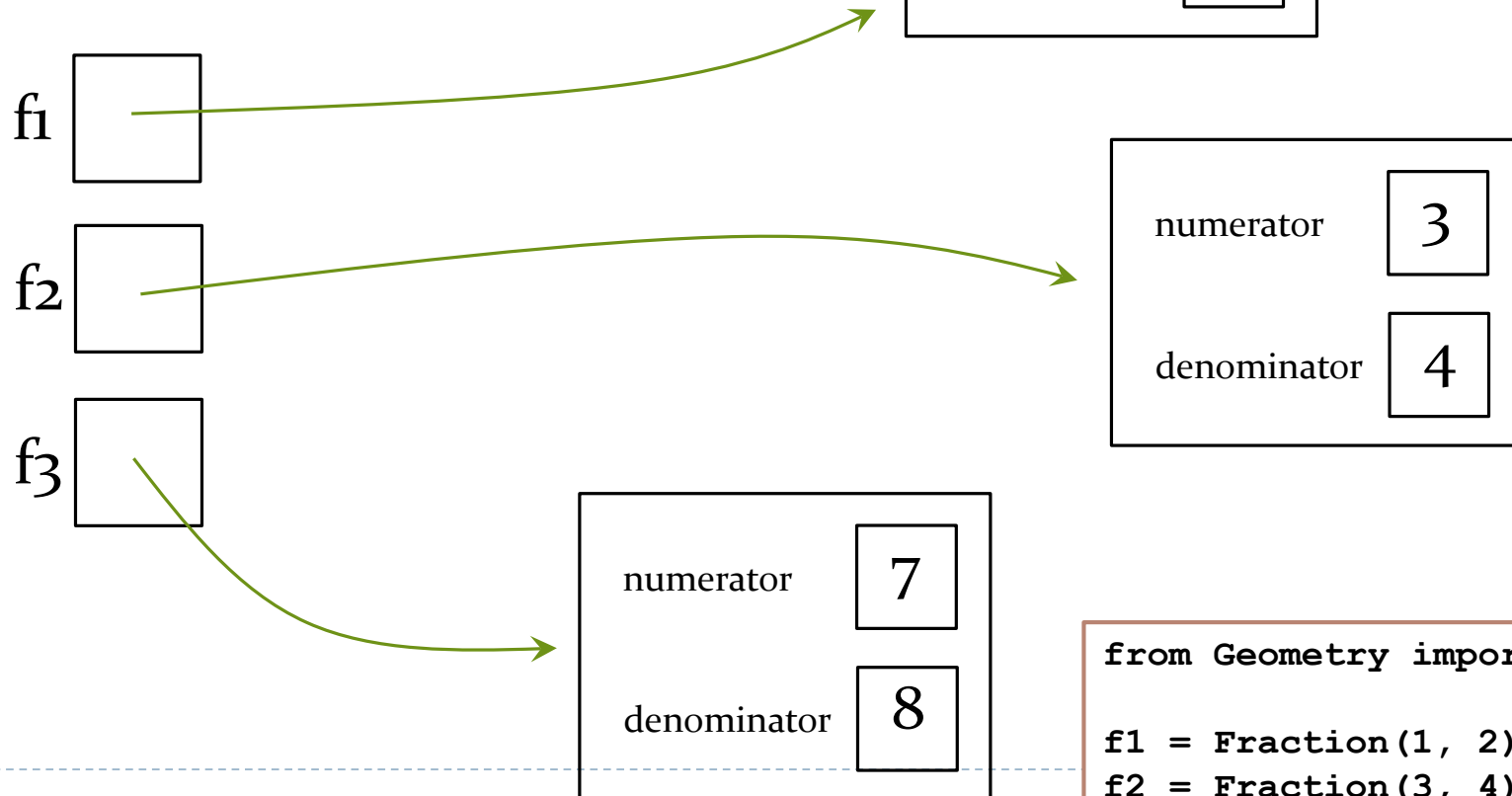
numerator

1 / 2

denominator

Example: A class for fractions

- ▶ We could create several Fraction objects as follows



main.py

```
from Geometry import Fraction  
  
f1 = Fraction(1, 2)  
f2 = Fraction(3, 4)  
f3 = Fraction(7, 8)
```



Initializer for the Fraction class

- ▶ The initializer for the Fraction class should store the numerator and the denominator as data fields.

```
class Fraction:
    def __init__(self, numerator, denominator):
        self.numerator = numerator
        self.denominator = denominator
```

```
f1 = Fraction(1, 2)
```

- ▶ So far, we can create a Fraction object

- ▶ However, if we want to create an object using:

```
f12 = Fraction()
```

- ▶ We need default values. e.g. default value for numerator is 0, and denominator is 1.

```
class Fraction:
    def __init__(self, numerator = 0, denominator = 1):
        self.numerator = numerator
        self.denominator = denominator
```

- ▶ What else can we do with Fractions?

- ▶ Nothing yet, we need to write more methods first!



Overriding default behavior

- ▶ All classes in Python get a number of special methods that are provided by default.
 - ▶ This includes e.g. methods for creating text-based representations (strings) of the object and methods for comparing two objects of the class type.

```
from Geometry import Fraction

f1 = Fraction(1, 2)
f2 = Fraction(1, 2)

print(f1) Calls the special function “__str__”

print(f1 == f2)
```

But these default versions are not very useful. We should define our own that make more sense for our class.

Calls the special function “__eq__”

```
<Fraction.Fraction object at 0x025E7410>
False
```



Exercise 3

- ▶ Consider the class definition:

```
class Fraction:
    def __init__(self, numerator = 0, denominator = 1):
        self.numerator = numerator
        self.denominator = denominator
    def __str__(self) ...
```

- ▶ What is the output of the code fragment below?

```
f1 = Fraction(20, 2)
f2 = Fraction()
print(f1.numerator, end=' ')
print(f2.denominator, end=' ')
print(f2)
```

__repr__

- ▶ The `__repr__` method produces a string that unambiguously describes the object
 - ▶ All classes should have a `__repr__` function implemented
 - ▶ Ideally, the representation could be used to create the object
 - ▶ For example, a fraction created using `Fraction(2, 3)` should have a `__repr__` method that returned `'Fraction(2, 3)'`

```
class Fraction:
    def __init__(self, numerator = 0, denominator = 1):
        self.numerator = numerator
        self.denominator = denominator
    def __repr__(self):
        return 'Fraction({}, {})'.format(self.numerator, self.denominator)
```

```
f1 = Fraction(2, 3)
print(repr(f1))
```

`Fraction(2, 3)`

Without the `__repr__` method

```
f2 = Fraction(4, 5)
print(repr(f2))
```

`<__main__.Fraction object
at 0x02762290>`

__str__

- ▶ The `__str__` method returns a string representing the object
 - ▶ By default, it calls the `__repr__` method!
 - ▶ The `__str__` method should focus on being human readable and nicely formatted

```
def __str__(self):  
    return str(self.numerator) + '/' + str(self.denominator)
```

- ▶ After we have implemented the method, we can use the print function to print the object

```
f1 = Fraction(3, 4)  
print(f1)
```

3/4

Without the `__str__`
method

<__main__.Fraction object
at 0x02762290>)



str and repr

- ▶ What is the difference between the `__str__` and `__repr__` methods of a Python object?
 - ▶ In short: the goal for `__repr__` is to be **unambiguous** and the goal for `__str__` is to be **readable**
 - ▶ The official Python documentation says:
 - `__repr__` is used to compute the “official” string representation of an object and
 - `__str__` is used to compute the “informal” string representation of an object.
 - ▶ The `print` statement and the `str()` built-in function uses `__str__`
 - ▶ The `repr()` built-in function uses `__repr__` to construct a string

```
p1 = Point(3, 4)
print(repr(p1))
print(p1)
```

Point(3, 4)
(3, 4)

```
f1 = Fraction(10, 1)
print(repr(f1))
print(f1)
```

Fraction(10, 1)
10/1

```
s1 = Square(10)
print(repr(s1))
print(s1)
```

Square(10)
10 x 10 Square

Exercise 4-5

► Q4: Continuing on from the class definition for a square

- Modify the initializer such that the initializer creates a square object with default value of 10
- Define a method named `__repr__(self)`
- Define a method named `__str__(self)`

```
s1 = Square()  
print(repr(s1))  
print(s1)
```

```
Square(10)  
10 x 10 Square
```

► Q5: Circle class

- a private data field named “radius”
- an initializer (default radius = 1)
- a method named `get_diameter(self)`
- `__str__`, `__repr__`

```
c1 = Circle(10)  
c2 = Circle()  
  
print(c1)  
print(c2)  
print(repr(c1))  
print(repr(c2))
```

```
A circle with a radius of 10cm  
A circle with a radius of 1cm  
Circle(10)  
Circle(1)
```

Exercise 6

- ▶ Consider the Point class defined previously

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

- ▶ Define a method named *halfway(target)*
 - ▶ takes a Point as an argument and returns the halfway point between itself and the parameter Point.
 - ▶ Note: the return type must be a Point object
- ▶ For example:

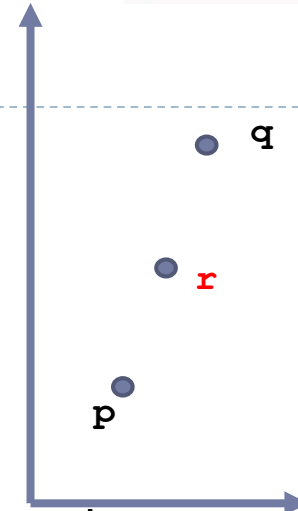
```
def get_halfway_point(self, target):

    mx =                /2

    my =                /2

    return Point(mx, my)
```

Must create a point and
then return it



```
p = Point(3, 4)
q = Point(5, 12)
r = p.get_halfway_point(q)
print(type(r))
print(r.x)
print(r.y)
```

```
<class '__main__.Point'>
4.0
8.0
```



Overloading Operators

- ▶ Python operators work for built-in classes.
- ▶ But same operator behaves differently with different types.
 - ▶ E.g. the `+` operator:
 - perform arithmetic addition on two numbers,
 - merge two lists
 - concatenate two strings.
- ▶ Allowing the same operator to have different meaning according to the context is called **operator overloading**

Operator	Expression	Internally
Addition	<code>f1 + f2</code>	<code>f1.__add__(f2)</code>
Subtraction	<code>f1 - f2</code>	<code>f1.__sub__(f2)</code>
Equality	<code>f1 == f2</code>	<code>f1.__eq__(f2)</code>
...		

__add__

- ▶ The `__add__` method is called when the `+` operator is called
- ▶ If we implement `__add__` then we can use `+` to add the objects
 - ▶ `f1 + f2` gets translated by Python into `f1.__add__(f2)`

```
def __add__(self, other):  
    new_num = self.numerator * other.denominator +  
              self.denominator * other.numerator  
    new_den = self.denominator * other.denominator  
    return Fraction(new_num, new_den)
```

$= 1/2 + 1/4$
 $= 4/8 + 2/8$
 $= 6/8$

must return a new
Fraction object

```
f1 = Fraction(1, 2)  
f2 = Fraction(1, 4)  
f3 = f1 + f2  
print(z)
```

6/8

`__eq__` (Version 1)

- ▶ The `__eq__` method checks equality of the objects
 - ▶ Default behaviour is to compare the references
 - ▶ But: In general we want to **compare the contents!**

```
def __eq__(self, other):  
    return self.numerator * other.denominator ==  
           other.numerator * self.denominator
```

```
f1 = Fraction(2, 5)  
f2 = Fraction(2, 5)  
print (f1 == f2)
```

True

```
f3 = Fraction(12, 30)  
f4 = Fraction(2, 5)  
print (f3 == f4)
```

True

```
f5 = Fraction(4, 10)  
f6 = Fraction(2, 5)  
print (f5 == f6)
```

True

```
f1 = Fraction(1, 2)  
f2 = Fraction(1, 4)  
print (f1 == f2)
```

False

```
f1 = Fraction(2, 3)  
print (f1 == 2)
```

AttributeError:
...

but if we compare a Fraction object with an integer, we will get an error message



Improving `__eq__`

- ▶ Check the type of the other operand
 - ▶ If the type is not a Fraction then not equal!

```
def __eq__(self, other):  
    if not isinstance(other, Fraction):  
        return False  
    return self.numerator * other.denominator ==  
           other.numerator * self.denominator
```

```
f1 = Fraction(2, 3)  
print (f1 == 2)
```

False

Other standard Python operators

- ▶ Many standard operators and functions:

<https://docs.python.org/3.8/library/operator.html>

- ▶ Common Arithmetic operators

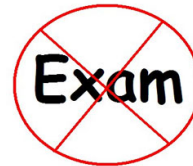
- ▶ `object.__add__(self, other)`
- ▶ `object.__sub__(self, other)`
- ▶ `object.__mul__(self, other)`
- ▶ `object.__truediv__(self, other)`

- ▶ Common Relational operators

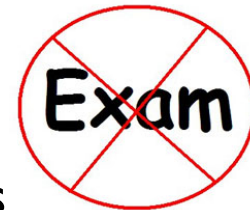
- ▶ `object.__lt__(self, other)`
- ▶ `object.__le__(self, other)`
- ▶ `object.__eq__(self, other)`
- ▶ `object.__ne__(self, other)`
- ▶ `object.__gt__(self, other)`
- ▶ `object.__ge__(self, other)`

Inplace arithmetic operators

- `object.__iadd__(self, other)`
- `object.__isub__(self, other)`
- `object.__imul__(self, other)`
- `object.__itruediv__(self, other)`



`+=`
`-=`
`...`



Reversed versions

- `object.__radd__(self, other)`
- `object.__rsub__(self, other)`
- `object.__rmul__(self, other)`
- `object.__rdiv__(self, other)`
- ...

Exercise 7 __eq__

- ▶ Consider the Point class defined previously

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

- ▶ Define the __eq__ method which defines the behaviour of the equality operator.
 - ▶ two points are considered to be equal to each other if and only if they have exactly the same position.
 - ▶ return True if two points are equal, and False otherwise.

```
def __eq__(self, other):
```

```
p = Point(3, 4)
q = Point(5, 12)
r = Point(3, 4)
print(p == q)
print(p == r)
print(p == 2)
print(p == (2, 4))
```

```
False
True
False
False
```


Exercise 8 __add__

- ▶ Consider the Point class defined previously

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

- ▶ Define the __add__(self, other) method
 - ▶ takes a point as a parameter and adds the two points together.
 - ▶ returns a new point with their sum of x-coordinates as well as their y-coordinates.

```
def __add__(self, other):

return Point( ... )
```

must return a new
Point object

```
p1 = Point(10, 20)
p2 = Point(30, 40)
p3 = p1 + p2
print("p1 =", p1)
print("p2 =", p2)
print("p3 =", p3)
```

p1 = (10, 20)
p2 = (30, 40)
p3 = (40, 60)



Instance Methods

- ▶ Instance methods are functions built in to the class definition of an object and require an instance of that class to be called.
- ▶ Instance methods may call other methods in their definitions
- ▶ e.g.
 - ▶ the `get_perimeter()` method

```
class Circle:
    def __init__(self, radius = 1):
        self.radius = radius

    ...
    def get_diameter(self):
        return self.radius * 2
    def get_perimeter(self):
        return self.get_diameter() * math.pi
```

Exercise 9

- ▶ Continuing on from the Fraction class,
 - ▶ implementing the special `__truediv__(self, other)` method.
 - ▶ is called automatically by Python whenever the `/` operator is used on a Fraction object.
 - ▶ "`f1 / f2`" $\Rightarrow$ `f1.__truediv__(f2)`
 - ▶ The `__truediv__(self, other)` method takes a fraction as a parameter and returns the `f1 / f2`.

```
class Fraction
...
def __truediv__(self, other):

    return Fraction(...)
```

```
= 1/3 / 4/5
= 1/3 * 5/4
= 5/12
```

```
a = Fraction(1, 3)
b = Fraction(4, 5)
d = a / b
print(d)
print(type(d))
```

```
5/12
<class
'__main__.Fraction'>
```



Summary

- ▶ A class is a template, a blueprint and a data type for objects
- ▶ A class defines the data fields of objects, and provides an initialiser/constructor for initialising objects and other methods for manipulating data
- ▶ The initialiser is always called `__init__()`
 - ▶ The first input parameter is always “self”, and this refers to the object being constructed
- ▶ We can override the default methods in a class definition
 - ▶ Such as `__repr__` and `__str__` and `__eq__`